

Trabalho Prático 06 - Multiplicação de Matrizes com Paralelismo

Especificações da Máquina utilizada:

O processador utilizado para o desenvolvimento dos testes foi o Ryzen 7 4800H, um Octa-Core que possui 16 Threads, com velocidade base do clock de 2.9 GHz e memórias cache L1, L2 e L3 com 512 KB, 4 MB e 8 MB respectivamente. O sistema operacional utilizado foi o Windows 11 Home x64 (64 bits).

Questões:

1) Questão

Tabela:

Ordem Matriz	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
500	0.313	0.155	0.079	0.061	0.049	0.052
1000	3.780	1.906	0.977	0.640	0.661	0.685
1500	14.867	7.257	3.761	2.364	3.509	3.544
2000	59.110	39.595	25.565	14.996	12.865	12.634
2500	87.823	37.869	19.278	10.153	12.084	13.796

Tabela 01 - Tabela com os tempos médios (em segundos) de execução do algoritmo de Multiplicação Matricial paralelizada.

Para a obtenção dos dados da tabela acima, foram realizadas 20 execuções do programa para cada Ordem da Matriz X Quantidade de Threads e foi calculada a média aritmética do tempo de execução da Multiplicação Matricial para cada um.

A seguir, serão apresentados os gráficos com as relações entre Ordem Matriz X Número de Threads e o tempo, além de mostrar o desvio padrão para cada barra do gráfico.

Gráficos:

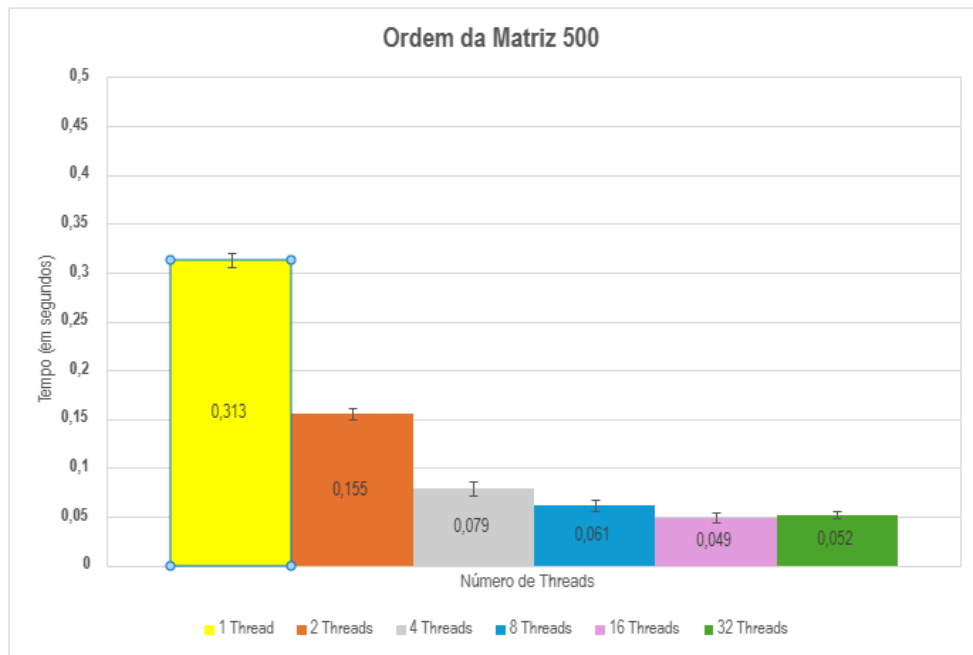


Figura 01- Gráfico que corresponde a relação entre ordem da Matriz 500, número de threads e o tempo de execução do programa (O tracinho em formato de 1 romano é o desvio padrão). O tempo vai de 0 a 0.5 segundos e a quantidade de threads testadas são 1,2,4,8,16 e 32 (o desvio padrão apresentado foi relativamente significativo).

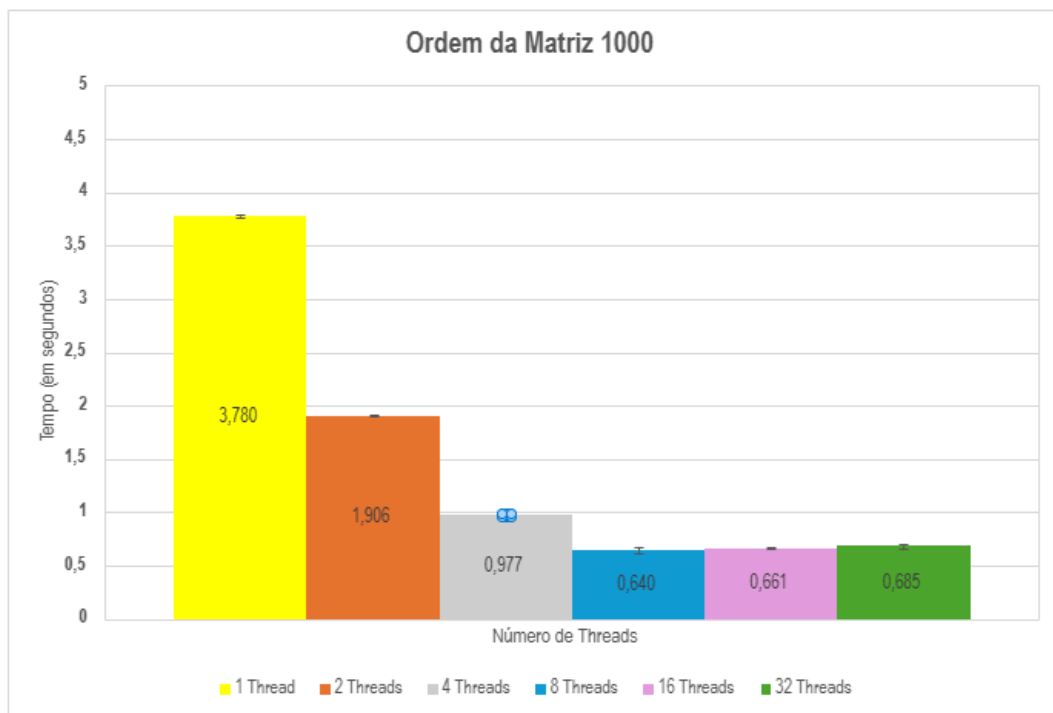


Figura 02- Gráfico que corresponde a relação entre ordem da Matriz 1000, número de threads e o tempo de execução do programa. O tempo vai de 0 a 5 segundos e a quantidade de threads testadas são 1,2,4,8,16 e 32 (desvio padrão pouco significativo).

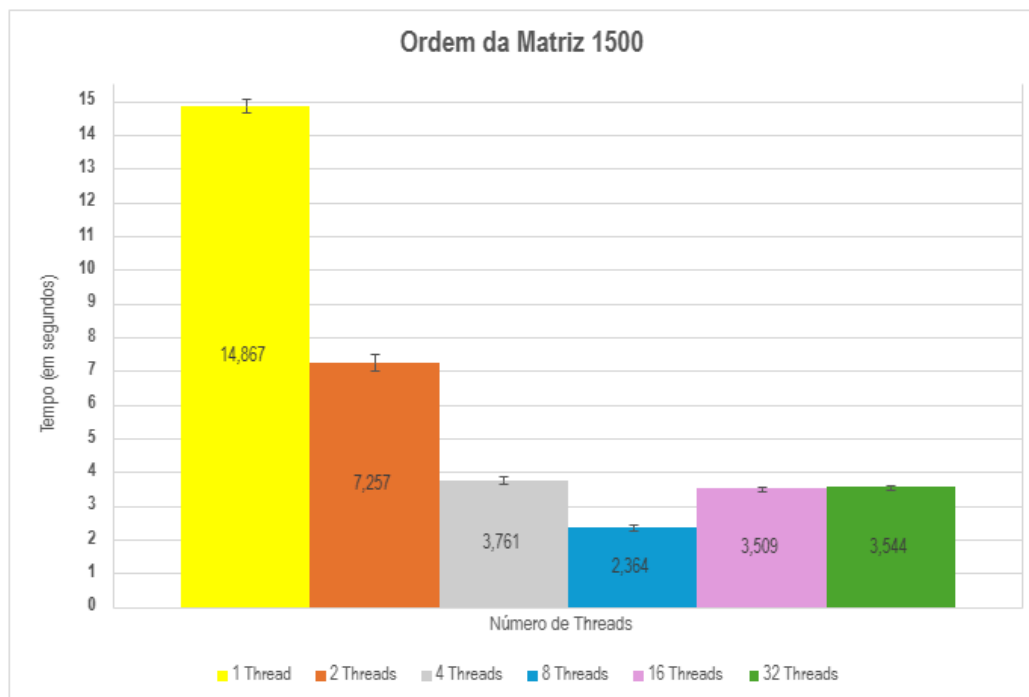


Figura 03- Gráfico que corresponde a relação entre ordem da Matriz 1500, número de threads e o tempo de execução do programa. O tempo vai de 0 a 15 segundos e a quantidade de threads testadas são 1,2,4,8,16 e 32 (apresentou um pequeno desvio padrão).

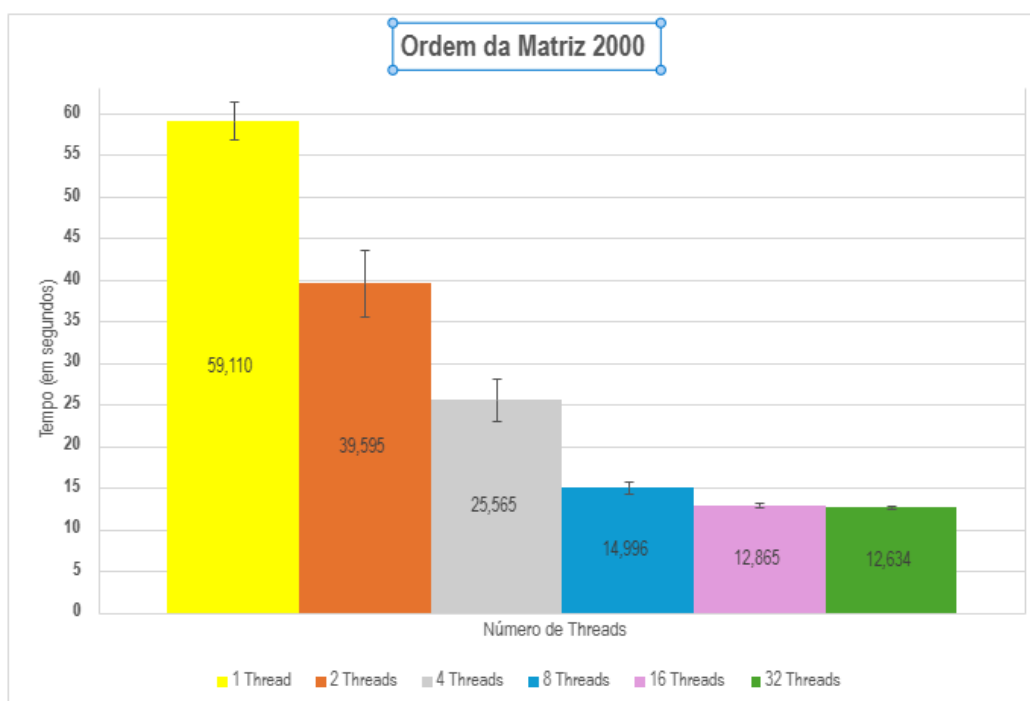


Figura 04- Gráfico que corresponde a relação entre ordem da Matriz 2000, número de threads e o tempo de execução do programa. O tempo vai de 0 a 60 segundos e a quantidade de threads testadas são 1,2,4,8,16 e 32 (apresentou um desvio padrão grande nos Threads 1,2 e 4).

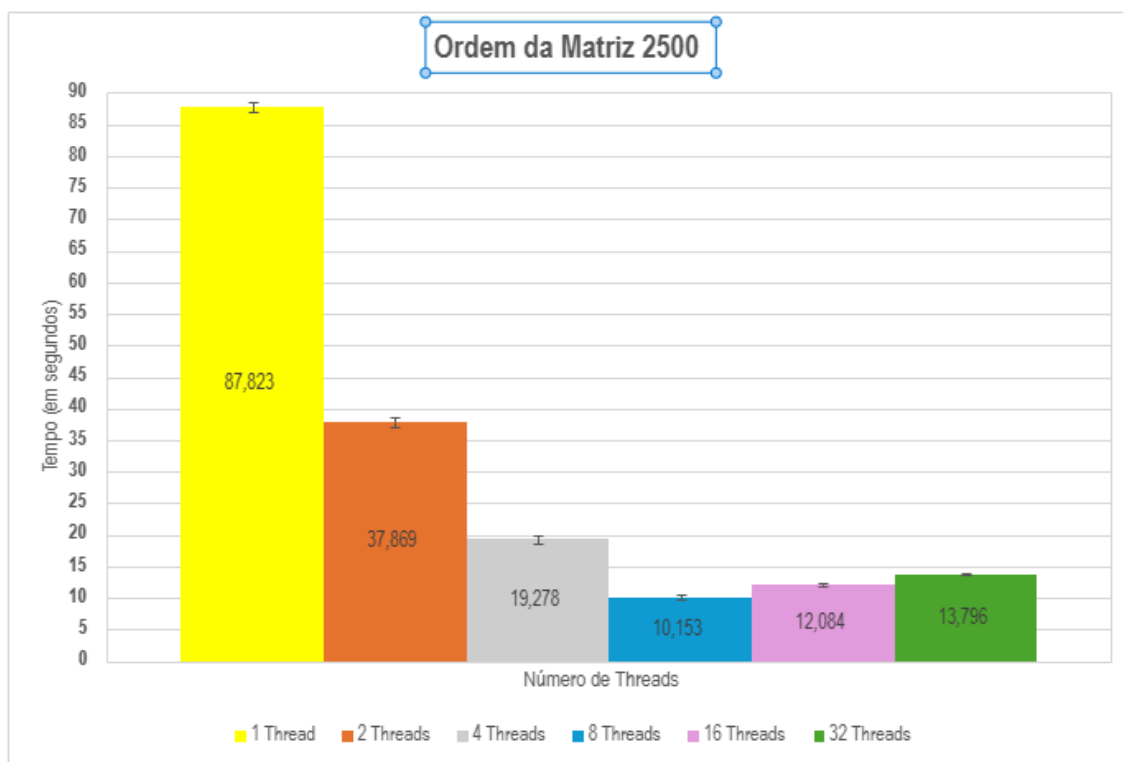


Figura 05- Gráfico que corresponde a relação entre ordem da Matriz 2000, número de threads e o tempo de execução do programa. O tempo vai de 0 a 90 segundos e a quantidade de threads testadas são 1,2,4,8,16 e 32 (desvio padrão pouco significativo).

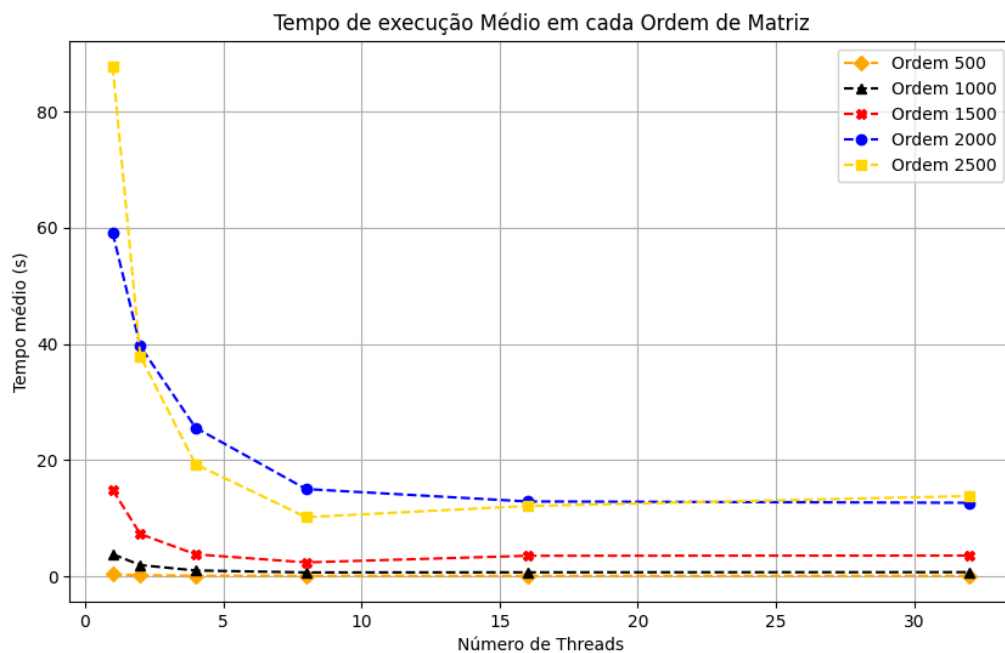


Figura 06- Gráfico que mostra a relação entre o número de Threads(1-32) e o tempo de execução em segundos para cada ordem de matriz (500,1000,1500,2000 e 2500). Esse gráfico foi feito em linhas para uma melhor visualização do comportamento.

2) Questão

Nas matrizes de Ordem 500 e 2000, a quantidade que apresentou a melhor média de tempo foi a de 16 threads, já nas demais Ordens (1000, 1500 e 2500) a quantidade que apresentou a melhor média foi a de 8 threads. Analisando os gráficos obtidos, é possível observar um grande salto de velocidade na execução a partir de 2 Threads, e essa diferença cresce bastante de uma para outra, até chegar em 8 (a metade da quantidade de Threads da CPU). Em relação a ordem da Matriz, quanto maior a Ordem, mais processamento vai ser necessário para realizar os cálculos e mais Threads causam um grande impacto no tempo de execução, diminuindo praticamente pela metade de 1 para 2, 2 para 4 e 4 para 8 (metade de muito, também é muito, logo é muito útil a paralelização nesses casos). Em suma, o número de Threads ideal é entre 8-16 (metade do número de Threads da Cpu e o número cheio), mais que isso há mais desvantagens do que vantagens e quanto maior a matriz, melhor será o ganho com a paralelização.

3) Questão

O aumento do número de Threads apresenta um ganho significativo quando a Ordem da Matriz é muito grande e/ou quando o número de Threads utilizado é menor ou igual ao número de Threads da CPU. Já situações que não apresentam um ganho de desempenho significativo é quando o número de Threads extrapola a quantidade de Threads da CPU, gerando mais trabalho para gerenciar memória e recursos do sistema, podendo deixar o programa mais lento devido a alternância das Threads. Outra desvantagem é quando a matriz é muito pequena e não vale tanto a pena paralelizar (muito trabalho para pouca diferença).

4) Questão

Dependendo da implementação do algoritmo, um dos possíveis overheads seria o acesso a memória compartilhada, muitas threads acessando o mesmo espaço praticamente ao mesmo tempo poderia causar conflito na hora de executar a sua função (aconteceu em uma das versões que implementei, as Threads recebiam e acessava o mesmo endereço da struct e precisava de alguns campos dela para realizar os cálculos, e como a execução das outras Threads também se iniciavam, a função recebiam parâmetros diferentes do desejado, causando problemas na execução). Outro overhead possível dependeria da quantidade de Threads criadas, se fosse muito maior que a Ordem da matriz, o sistema iria passar mais tempo gerenciando as threads do que de fato multiplicando, tornando o programa mais lento. Um outro overhead seria o custo de recursos e memória do sistema, já que o sistema operacional precisa alocar várias coisas para as Threads funcionarem. Outro overhead é a espera de todas as threads finalizarem para obter o produto matricial das duas matrizes (também depende da implementação).

